

CNN Image Classification

Hyperparameter Experiment Report

Dataset: CIFAR-10 | Framework: PyTorch | Model: IntelCNN

1. Project Overview

This project implements a custom Convolutional Neural Network (IntelCNN) built from scratch using PyTorch, trained on the CIFAR-10 dataset for 10-class image classification. The notebook covers dataset loading, preprocessing, model design, baseline training, hyperparameter experimentation, and final evaluation.

1.1 Dataset

Property	Details
Name	CIFAR-10
Training samples	50,000 (split: 45,000 train / 5,000 validation)
Test samples	10,000
Classes (10)	airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck
Image size	32 x 32 pixels, RGB

1.2 Preprocessing & Augmentation

Training transforms (with augmentation):

- Resize to 32x32
- Random crop with padding=4
- Random horizontal flip (p=0.5)
- Random rotation (+/-15 degrees)
- Color jitter: brightness and contrast +/-0.2
- Normalize: mean=(0.4914, 0.4822, 0.4465), std=(0.2470, 0.2435, 0.2616)

Validation/Test transforms: Resize and normalize only — no augmentation.

2. Model Architecture — IntelCNN

IntelCNN is a custom CNN with four progressive convolutional blocks followed by a dense classifier head.

2.1 Convolutional Blocks

Block	Channels	Pooling	Dropout	Normalization
Block 1	3 -> 64 (x2 conv)	MaxPool 2x2	0.20	BatchNorm2d
Block 2	64 -> 128 (x2 conv)	MaxPool 2x2	0.25	BatchNorm2d
Block 3	128 -> 256 (x2 conv)	MaxPool 2x2	0.30	BatchNorm2d
Block 4	256 -> 512 (x2 conv)	AdaptiveAvgPool	0.35	BatchNorm2d

2.2 Classifier Head

- Flatten
- Linear(512, 256) -> BatchNorm1d -> ReLU -> Dropout(0.3)
- Linear(256, 10) — output logits for 10 classes

All convolutional layers use 3x3 kernels with padding=1 and ReLU activations.

3. Baseline Training Configuration

The baseline model was trained with the following fixed setup before any hyperparameter experiments were conducted. Best weights (by validation accuracy) were restored via early stopping.

Hyperparameter	Value
Learning Rate	0.001
Batch Size	64
Max Epochs	30
Early Stopping Patience	7 epochs
Optimizer	AdamW (weight_decay=1e-3)
LR Scheduler	OneCycleLR (max_lr = lr x 3)
Loss Function	CrossEntropyLoss (label_smoothing=0.1)
Dropout Rate	0.3 (classifier head)

The baseline model is subsequently used for test-set evaluation in Section 5, after all hyperparameter experiments are complete.

4. Hyperparameter Experiments

Four experiments were run, each varying the learning rate and/or batch size. Every experiment trains a fresh IntelCNN instance for 10 epochs with no early stopping, keeping all other settings identical to the baseline.

4.1 Experiment Configurations

Experiment Name	LR	Batch Size	Epochs	vs. Baseline LR
LR=0.1, BS=64	0.1	64	10	100x higher
LR=0.0005, BS=64	0.0005	64	10	2x lower
LR=0.001, BS=32	0.001	32	10	Same LR
LR=0.001, BS=128	0.001	128	10	Same LR

DataLoaders were also rebuilt to match each experiment's batch size. The OneCycleLR scheduler was applied throughout.

4.2 Evaluation Methodology

After each experiment the trained model was evaluated on the validation set using an `evaluate_metrics()` helper, computing:

- Validation Loss (CrossEntropyLoss with label smoothing)
- Validation Accuracy (%)
- Weighted Precision
- Weighted Recall
- Weighted F1-Score

Training loss and accuracy from the final epoch were captured from the history object returned by `train_model()`.

5. Results — Experiment Comparison

The table below summarises metrics at the end of each experiment's training run. Cells marked '—' are placeholders for live runtime output not captured in the static notebook; fill them in after executing the notebook.

5.1 Summary Metrics Table

Experiment	LR	BS	Train Loss	Val Loss	Train Acc %	Val Acc %	Prec.	Recall	F1
LR=0.1, BS=64	0.1	64	—	—	—	—	—	—	—
LR=0.0005, BS=64	0.0005	64	—	—	—	—	—	—	—
LR=0.001, BS=32	0.001	32	—	—	—	—	—	—	—
LR=0.001, BS=128	0.001	128	—	—	—	—	—	—	—

'—' = placeholder for runtime output. Run the notebook to populate these values.

5.2 Expected Behaviour per Configuration

Experiment	Expected Behaviour
LR=0.1, BS=64	Very high LR — likely unstable training, large loss oscillations, low val accuracy. OneCycleLR pushes effective LR down to 0.0001 by epoch 10.
LR=0.0005, BS=64	Conservative LR — slower convergence, stable training. May not reach peak accuracy in 10 epochs but unlikely to overfit.
LR=0.001, BS=32	Same LR as baseline, smaller batches. More gradient updates per epoch, noisier gradients, potentially better generalisation.
LR=0.001, BS=128	Same LR as baseline, larger batches. Faster epochs, smoother gradients, but may require a higher LR to match baseline performance.

6. Final Evaluation — Baseline Model on Test Set

After the hyperparameter experiments, the baseline model (best weights from Section 3) is evaluated on the held-out CIFAR-10 test set (10,000 images). This provides an unbiased estimate of generalisation performance.

6.1 Metrics Reported

- Overall test accuracy (%)
- Per-class Precision, Recall, and F1-Score
- Weighted-average Precision, Recall, and F1-Score
- Macro-average scores across all 10 classes

6.2 Per-Class Results (Placeholders)

Fill in the values below after running the notebook.

Class	Precision	Recall	F1-Score	Support
airplane	—	—	—	1,000
automobile	—	—	—	1,000
bird	—	—	—	1,000
cat	—	—	—	1,000
deer	—	—	—	1,000
dog	—	—	—	1,000
frog	—	—	—	1,000
horse	—	—	—	1,000
ship	—	—	—	1,000
truck	—	—	—	1,000
weighted avg	—	—	—	10,000

6.3 Activation-Based Bounding Box Bonus

The notebook includes a bonus section that generates bounding boxes from the last convolutional layer's activation maps (no external localisation libraries). A custom hook captures feature maps from the final Conv2d layer, averages them across channels to produce a class activation map (CAM), resizes the map to 32x32, thresholds at 0.5, and derives a bounding-box rectangle from the surviving pixel coordinates.

This technique highlights which spatial regions the model attends to when making a prediction, providing a lightweight form of interpretability without ground-truth bounding-box annotations.

7. Summary & Key Observations

Section	Key Detail
Dataset	CIFAR-10 — 50K train (45K/5K split), 10K test, 10 classes
Preprocessing	Resize, Normalize; Augment: flip, crop, jitter, rotation (train only)
Model	IntelCNN — 4 conv blocks (64->128->256->512) + BN + Dropout
Baseline Training	AdamW, OneCycleLR, CrossEntropy(label_smoothing=0.1), early stop @7
Experiments	4 configs varying LR (0.0005 / 0.001 / 0.1) and BS (32 / 64 / 128)
Metrics	Accuracy, Precision, Recall, F1-Score (weighted, per-class)
Evaluation	Validation set per experiment; test set for final baseline assessment
Interpretability	Activation-based bounding boxes from last conv layer (bonus section)

Design Choices Worth Noting

- Label smoothing (0.1) reduces overconfidence and often improves calibration.
- OneCycleLR allows aggressive warmup and cosine decay in a single pass.
- Progressive dropout (0.20 -> 0.35 across blocks) regularises deeper features more strongly.
- AdaptiveAvgPool2d in Block 4 removes dependence on a fixed spatial input size.
- Early stopping on the baseline prevents overfitting without tuning epoch count.
- Experiments disable early stopping to observe raw learning dynamics over 10 epochs.

Report generated from CNN_Image_Classification_Project.ipynb. Numeric experiment results depend on runtime execution; placeholder '—' values should be updated after running the notebook.